Tutorial 03: Auction Inventory Analytics

شرح كامل من الصفر بالمصري: Python + SQL Server + Power BI

مزادين Liquidation في Phoenix (LocalAuctions.com) | 168 lot | 41 buyer | Hammer = $3,710.02

المنتور بتاعك بيشرحلك عشان تعيد بناء المشروع بنفسك وتعرف تتكلم عنه في أي Interview

كل الأرقام في الدوسية اتراجعت بـ sqlcmd على الداتابيز Auction_Inventory قبل الكتابة

فهرس الدوسية

1. فكرة المشروع وليه مهم
2. الداتا: الـ Excel exports والتنضيف والـ schema والـ grain
3. إعداد البيئة: SQL Server Express والـ BULK INSERT
4. Python: الـ regex سطر بسطر والـ pandas
5. الـ SQL: ملف 00 (الـ view) وكل query من 01 لـ 06
6. Power BI: الموديل وكل الـ 15 DAX measure والـ visuals
7. Checklist: إزاي تبني المشروع من الصفر
8. Gotchas: الفخاخ اللي ممكن توقعك
9. Interview Prep: 14 سؤال بإجابات إنجليزي والاستراتيجية بالمصري

قبل ما نبدأ: كلمة صراحة

المشروع ده **عينة صغيرة وثابتة**: مزادين في يوم واحد (28 مارس 2026) في مدينة واحدة. مفيش أي داتا عن **التكلفة** (cost) ولا الـ fees، فمفيش **margin** حقيقي. اللي بنقيسه هو الـ **hammer price** (سعر المطرقة النهائي) مقارنة بالـ **retail price المعلن** في عنوان الـ lot، وده السعر اللي البايع كاتبه، مش سعر السوق المؤكد. أي جملة في الدوسية أو الـ interview لازم تتقال في حدود ده.

ملاحظة مهمة عن الكود الفعلي

ملف `requirements.txt` فيه `pandas` بس السكريبت `auction_analysis.py` الفعلي **مبيستخدمش pandas**. بيستخدم `openpyxl` و `csv` و `re` من الـ standard library. هنشرح الكود الحقيقي الأول وبعدين هنكتب المقابل بـ pandas كمان عشان انت لسه بتتعلمه. وكمان الـ `category` بيتحسب في **SQL (الـ view)** مش في Python.

1) فكرة المشروع وليه مهم

1.1 القصة ببساطة

في مزادات الـ liquidation، شركات بتصفّي بضاعة (مرتجعات، فائض مخازن، أجهزة كهربا) والناس بتزايد عليها. كل **lot** (قطعة أو مجموعة) بيبدأ بـ **opening bid** بخمس دولار، وبيتباع لأعلى مزايد. السعر النهائي اسمه **hammer price**.

السؤال الأساسي للمشروع: **"الـ lots دي بتتباع بكام، بقى أد إيه تحت الـ retail المعلن، ومين اللي بيشتري؟"**

1.2 النتايج الرئيسية (كلها متراجعة)

المقياس	القيمة	المصدر
عدد الـ lots	168 (162 + 6)	query 01
إجمالي الـ hammer	$3,710.02	query 01
الـ median للـ lot	$5.00	query 01
الـ mean	$22.08	query 01
أعلى lot	$240.01 (ثلاجة Vissani، lot 28142)	query 04
Lots اتباعت بـ $5 بالظبط	96 (57.1%)	query 01/02
Lots فيها retail معلن	120 من 168 (48 من غير)	view
Retail إجمالي / Hammer على نفس الـ 120	$27,838.65 / $3,320.02	query 03
Recovery (سنت لكل retail دولار)	11.9% يعني 11.9 سنت، والـ discount 88.1%	query 03
Unique buyers	41	query 01
أكبر buyer / أكبر 5	22.5% ($835) / 60.8%	query 05
أرخص category (recovery)	Range Hoods: 4.5%	query 06
أكبر فجوة دولارية	Refrigeration & Ice: $5,994.46 على 13 lot فيهم retail	query 06

1.3 ليه المشروع ده مفيد في الـ portfolio؟

- بيوري الـ pipeline كامل: ملف Excel وسخ، Python، SQL Server، Power BI. ده بالظبط شغل الـ data analyst.
- بيوري **feature engineering من نص**: السعر مش عمود، هو جوه الـ title وبنطلعه بـ regex (في Python) وبـ PATINDEX (في SQL).
- بيوري **الأمانة التحليلية**: بتقول إيه اللي المشروع بيثبته وإيه اللي لأ.
- فيه قصة bug حقيقية (119 مقابل 120) بتتحكي في أي interview كمثال على الـ validation.

> **ليه "cents per retail dollar"؟**
>
> لأن "discount 88.1%" ممكن تتفهم غلط. الأوضح: "بتدفع **11.9 سنت** مقابل كل دولار retail". المفهوم ده اسمه Recovery Rate = hammer ÷ retail، والـ discount = 1 - recovery. الاتنين وجهين لنفس الرقم، بس الـ "سنت لكل دولار" أسهل للمدير يفهمه ويقارن بيه بين categories.

1.4 اللي المشروع مش بيقوله

- مش بيقول الـ reseller هيكسب كام: مفيش cost ولا fees ولا وقت ولا مصاريف نقل. وفي ملف الـ template (auction_23613_items_list.csv) الـ buyer's premium مكتوب 15% والـ sales tax 9.1%، يعني اللي المشتري بيدفعه فعلاً أعلى من الـ hammer.
- مش بيقول إن الـ retail المعلن "حقيقي": ده رقم كتبه البايع في الـ title.
- مش بيعمم: مزادين، يوم واحد، مدينة واحدة.

2) الداتا

2.1 الملفات

File	What
data/raw/home_kitchen_auction_23154_results.xlsx	Home & Kitchen auction, 162 lots
data/raw/pallets_auction_23166_results.xlsx	Pallets auction, 6 lots
data/raw/auction_23613_items_list.csv	consignment/listing template (13 rows), مش جزء من التحليل
data/home_kitchen_results_clean.csv, pallets_results_clean.csv	output Python

الـ Excel اللي نازل من الموقع مش جدول نضيف. لما قريته بـ openpyxl لقيت أول 8 صفوف (0 لـ 7) كلها أو تقريباً فاضية: اسم الشركة، العنوان (2225 W Peoria Ave Suite 200, Phoenix, AZ 85029)، رقم التليفون، اللينك، وبعدين عنوان "Auction Results for PHOENIX, AZ Liquidation Land Home and Kitchen Auction 3/28/2026 ID23154". بعد كده بييجي الـ header الحقيقي اللي أول خلية فيه `Lot #`.

2.2 الـ grain (مستوى التفاصيل)

> **Grain = صف واحد لكل lot اتباع**
>
> المفتاح الأساسي في جدول `lots` هو `(auction_id, lot)`. رقم الـ lot لوحده **مش كفاية** لأنه ممكن يتكرر بين مزادين مختلفين (ده أصلاً سبب إن الـ PK مركب). مهم تحدد الـ grain قبل أي query، لأن كل `COUNT` و `SUM` بتعتمد عليه.

2.3 الأعمدة بعد التنضيف

column	type in SQL	note
auction_id	INT	23154 or 23166, FK to auctions
lot	VARCHAR(10)	text, to keep leading zeros (0094..0099)
title	NVARCHAR(300)	often starts with "(Retail $...)"
buyer_id	VARCHAR(20)	anonymous bidder number
quantity	INT	
hammer_price	DECIMAL(10,2)	money: never FLOAT

2.4 خطوات التنضيف (ذهنياً)

1. **تجاهل الـ junk rows:** دوّر على الصف اللي أول خلية فيه "Lot #" وابدأ من اللي بعده.
2. **تجاهل الصفوف الفاضية** (الخلية الأولى None أو فاضية).
3. **خلّي الـ lot نص**: لو حولته رقم، "0094" هتبقى 94 وتضيع الأصفار (والـ 0094 pallets lots لـ 0099).
4. **خلّي الـ buyer_id نص**: هو معرّف مش رقم للحساب.
5. **حوّل quantity لـ int و hammer لـ float**، مع حماية من الفاضي (`if r[3] else 0`).
6. اكتب CSV نضيف، وبعدين SQL يقراه.

عينة من الـ CSV النضيف

```
lot,title,buyer_id,quantity,hammer_price
21922,(Retail $69) White Cordless Aluminum Mini Blinds 65&#34; W x 48&#34; L,139553,1,5.0
0094,Pallet of Assorted Items,225445,1,40.0
0097,Oven,10762,1,5.0
```

لاحظ: في الـ CSV في HTML entities زي `"` (علامة ")، وده بيتصلح في الـ view بـ `REPLACE` مش في Python. 9 lots في الملفات النضيفة فيها entities (اتحسبوا بـ Python).

3) إعداد البيئة

3.1 اللي محتاجه

- **SQL Server Express** بـ instance اسمه SQLEXPRESS\. (النقطة = الجهاز ده). الـ collation الافتراضي عندي `SQL_Latin1_General_CP1_CI_AS`، والـ **CI** معناها Case-Insensitive. ده مهم جداً في قصة الـ regex.
- **SSMS** أو sqlcmd لتشغيل الـ scripts. من الـ command line: `sqlcmd -S .\SQLEXPRESS -C -i 00_setup_database.sql` (الـ `-C` بتثق في الـ certificate).
- Python 3 + `pip install -r requirements.txt` (pandas و openpyxl).
- Power BI Desktop.

3.2 الـ BULK INSERT والمسار

الـ `BULK INSERT` بيتنفذ **من جوه SQL Server service** مش من حسابك انت. يعني اللي بيفتح الملف هو الـ service account (زي NT SERVICE\MSSQL$SQLEXPRESS). ده الـ account غالباً **معندوش صلاحية** على فولدر C:\Users\kamal\Downloads لأنه جوه الـ profile بتاعك.

> **ليه مش Downloads؟**
>
> لأن الـ service account مش بيقدر يدخل فولدر user profile، فتاخد error "Cannot bulk load. Operating system error code 5 (Access is denied)". الحل: انسخ الـ CSVs لفولدر مشترك زي C:\Users\Public\auction-inventory\ (ده المسار في الـ script وبيشتغل على default install). لو حطيتهم في مكان تاني، عدّل الـ path في الـ BULK INSERT (سطرين) أو اديه صلاحية قراءة.

3.3 ترتيب التشغيل

1. python analysis/auction_analysis.py (بيطلّع الـ clean CSVs في /data)
2. انسخ الـ CSVs لـ C:\Users\Public\auction-inventory\
3. شغّل sql/00_setup_database.sql (بيعمل الداتابيز والجداول والـ view، وآخره sanity check)
4. شغّل 01 لـ 06 واحد واحد
5. افتح الـ pbix وعمل Refresh (بيقرا من الـ view)

> **Sanity check لازم يطلع**
>
> **168 lots، hammer 3,710.02، و 120 lot فيهم retail_price.** لو طلع 119، يبقى فيه مشكلة case-sensitivity في مكان ما (القصة في الفصل 4).

4) Python: الـ regex والتنضيف

4.1 الصورة الكبيرة للسكريبت

الملف `analysis/auction_analysis.py` فيه 4 دوال: `load_results` (قراءة وتنضيف)، `retail_price` (regex)، `summarize` (إحصائيات لكل مزاد)، و `main` (يكتب CSV ويطبع النتايج). مفيش ML ولا مكتبات تقيلة.

4.2 الـ imports والثوابت

```
import csv, re
from collections import Counter
from pathlib import Path
from statistics import median
import openpyxl

ROOT = Path(__file__).resolve().parent.parent   # repo root, wherever you run from
RAW  = ROOT / "data" / "raw"
DATA = ROOT / "data"
```

`Path(__file__).resolve().parent.parent` : مكان الملف نفسه، طلعتين لفوق، فالسكريبت يشتغل من أي فولدر. ده أحسن من مسار نسبي بيعتمد على مكان تشغيلك.

4.3 دالة load_results سطر بسطر

```
def load_results(path):
    wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
    ws = wb.active
    rows = list(ws.iter_rows(values_only=True))
    header_idx = next(i for i, r in enumerate(rows) if r[0] == "Lot #")
    lots = []
    for r in rows[header_idx + 1:]:
        if r[0] is None or not str(r[0]).strip():
            continue
        lots.append({
            "lot": str(r[0]).strip(),
            "title": str(r[1]).strip() if r[1] else "",
            "buyer_id": str(r[2]).strip() if r[2] else "",
            "quantity": int(float(r[3])) if r[3] else 0,
            "hammer_price": float(r[4]) if r[4] else 0.0,
        })
    return lots
```

- `read_only=True` : بيقرا بسرعة وبذاكرة أقل. `data_only=True` : لو في formulas ياخد القيمة المحسوبة مش نص الـ formula.
- `values_only=True` : ترجعلي tuples من القيم بدل Cell objects.
- `next(i for ... if r[0] == "Lot #")` : generator بيلاقي **أول** index للـ header الحقيقي. لو مفيش، هيرمي `StopIteration` (يعني الشكل اتغير، وده كويس: يفشل بصوت عالي).
- الـ `continue` : بيتخطى الصفوف الفاضية (ممكن يبقى في صفوف تحت الجدول).
- `int(float(r[3]))` : لأن Excel ممكن يرجّع `1.0`، و `int("1.0")` بيفشل، فنعدي على float الأول.
- الـ `if r[3] else 0` : لو الخلية فاضية نحط 0 بدل ما ينهار.

تنبيه صغير: الـ encoding

في `main` الـ CSV بيتكتب بـ `open(out, "w", newline="")` من غير `encoding`. على Windows ده بيستخدم الـ code page الافتراضي (مش UTF-8 بالضرورة)، والـ `BULK INSERT` بتاعنا بيقرا بـ `CODEPAGE = '65001'` (UTF-8). لو عندك رموز غريبة في الـ titles (فيه lot فيه درجة حرارة 540 درجة)، الأمان إنك تكتب `encoding="utf-8"` صريح. ده تحسين مقترح، مش اللي في الكود حالياً.

4.4 الـ regex سطر بسطر

```
RETAIL_RE = re.compile(r"retail\s*\$([\d,]+\.?\d*)", re.IGNORECASE)

def retail_price(title):
    m = RETAIL_RE.search(title or "")
    return float(m.group(1).replace(",", "")) if m else None
```

piece	meaning
r"..."	raw string: backslashes are not Python escapes
retail	the literal word
\s*	zero or more whitespace (handles "Retail$69" and "Retail $69")
\$	literal dollar sign (plain $ means end-of-line in regex)
(...)	capture group 1: just the number
[\d,]+	one or more digits or commas (1,121)
\.?	optional decimal point
\d*	zero or more digits after (so "$69" without cents works)
re.IGNORECASE	match Retail, retail, REtail, RETAIL

`.search()` بيدوّر في أي مكان في الـ string (مش لازم من الأول زي `.match()`). `group(1)` هو الرقم بس. بنشيل الفاصلة (`"1,121.00"` يبقى `"1121.00"`) وبعدين float. لو مفيش match بنرجع `None` (مش 0)، لأن "مفيش retail" غير "retail بصفر".

مثال: `"(Retail $1,121.00) Lanbo 24 Inch Freestanding Electric Range"` يرجّع `1121.0`. و `"(Retail $69) White Cordless..."` يرجّع `69.0`.

قصة الـ bug: 119 مقابل 120

الـ title بتاع lot **28031** مكتوب `(REtail $119.99) ESR iPad Pro 13 Inch Case with Keyboard` (حرفين R و E كبار، والباقي صغير: غلطة كتابة عند البايع). الـ regex من غير `re.IGNORECASE` بحرف `Retail` الكبير كان بيفوّت الـ lot ده. النتيجة **119** lot بدل **120**، وده كان بيغيّر الـ retail total بمقدار $119.99.

اتأكدت منها بإيدي: إجمالي الـ titles اللي فيها كلمة retail = 120؛ pattern بحرف كبير `Retail` من غير flag بيلاقي 119؛ ومعاه `IGNORECASE` بيلاقي 120. ولوت 28031 بالتحديد retail = $119.99، hammer = $26.00. وده الـ lot الوحيد في الداتا بالـ casing الغريب. (بالنسبة للـ SQL: الـ `PATINDEX('%retail $%')` كان دايماً case-insensitive لأن الـ collation هو CI_AS، فالـ view كان بيرجّع 120 من الأول.)

الدرس: قارن العدّ بين الطبقات (Python مقابل SQL مقابل Power BI). لو الرقمين اتفرقوا بواحد، دوّر على الصف ده. وحوّل أي pattern على نص بشري لـ case-insensitive كأول قاعدة.

4.5 بدايل للـ regex

الطرق البديلة (pandas)

```
import pandas as pd

# 1) extract: returns the captured group (NaN if no match)
df["retail_price"] = (
    df["title"].str.extract(r"retail\s*\$([\d,]+\.?\d*)", flags=re.IGNORECASE)[0]
      .str.replace(",", "", regex=False)
      .astype(float)
)

# 2) contains: only boolean flag (how many lots have a retail price?)
has_retail = df["title"].str.contains(r"retail\s*\$", case=False, regex=True)
print(has_retail.sum())      # 120
```

- `str.extract` : الأنسب لما عايز الرقم. بيرجّع `NaN` لو مفيش، و `astype(float)` بيتعامل معاه.
- `str.contains(..., case=False)` : للفلترة والعدّ بس، مش بيطلّع الرقم.
- الـ regex في Python العادي بتبقى أبسط لو بتلف على list of dicts (زي الكود الحالي). في pandas الـ vectorized أسرع وأقصر على داتا كبيرة. بالـ 168 صف الفرق مش محسوس.
- بديل تالت: `SUBSTRING + PATINDEX` في SQL (وده اللي الـ view بيعمله، هنشرحه في الفصل 5).

(الكود ده توضيحي: الـ script الأصلي مش بيستخدم pandas.)

4.6 pandas cleaning: المقابل لـ load_results

```
raw = pd.read_excel(path, header=None, dtype=str)       # read everything as text
hdr = raw.index[raw[0] == "Lot #"][0]                    # row of the real header
df  = raw.iloc[hdr + 1:, :5].copy()
df.columns = ["lot", "title", "buyer_id", "quantity", "hammer_price"]
df = df[df["lot"].notna() & df["lot"].str.strip().ne("")]
df["quantity"]     = pd.to_numeric(df["quantity"]).fillna(0).astype(int)
df["hammer_price"] = pd.to_numeric(df["hammer_price"]).fillna(0.0)
df.to_csv(out, index=False, encoding="utf-8")
```

`dtype=str` هو السر في الحفاظ على الأصفار في أرقام الـ lots. لو سبت pandas يخمّن الـ "0094" types، هتبقى 94.

4.7 دالة summarize

- `median(prices)` من `statistics` : بيحسب الـ median بالـ interpolation بين الوسطين لو العدد زوجي (زي `PERCENTILE_CONT` بالظبط).
- القائمة `with_retail` بتتفلتر بـ `if r` يعني بتشيل `None` **وكمان 0** (retail صفر هيطلع خارج الحساب، وده كويس لأن القسمة على صفر).
- `retail_discount_pct = 100 * (1 - sum(h) / sum(r))` : ده **dollar-weighted** (مجموع على مجموع) مش متوسط الـ discounts لكل lot.
- `Counter(l["buyer_id"] ...)` بيعدّ الـ lots لكل buyer، و `spend[l["buyer_id"]] += price` بيجمع الإنفاق، و `most_common(1)` بيرجّع أكبر مُنفق.
- `max(lots, key=lambda l: l["hammer_price"])` : الـ lot الأعلى سعراً.

main 4.8

بيلف على المزادين، يكتب CSV لكل واحد بـ `csv.DictWriter` (الأعمدة: lot, title, buyer_id, quantity, hammer_price)، يطبع الملخص، وفي الآخر يطبع الإجمالي: 168 lot و $3,710.02.

> **دور Python في الـ pipeline**
>
> Python هنا بيعمل **الـ ingestion والتنضيف** وكمان بيحسب ملخص سريع كـ cross-check مستقل عن SQL. الـ retail و category و price bands اللي بتروح Power BI بتتحسب في **الـ view** بـ **SQL**. وده معناه إن عندك مصدرين لنفس الرقم (lot 120): لو اتفقوا، ثقتك أعلى.

5) الـ SQL كله

5.1 ملف 00: بناء الداتابيز والـ view

الملف ده الأهم لأنه بيعمل **كل الـ feature engineering**. هنقسمه أجزاء.

الجزء 1: الداتابيز والجداول

```
IF NOT EXISTS (SELECT 1 FROM sys.databases WHERE name = 'Auction_Inventory')
    CREATE DATABASE Auction_Inventory;
GO
USE Auction_Inventory;
GO
DROP VIEW  IF EXISTS dbo.v_lots;
DROP TABLE IF EXISTS dbo.lots;
DROP TABLE IF EXISTS dbo.auctions;
```

الـ **idempotent** script: تقدر تشغله 10 مرات والنتيجة واحدة. بنمسح بالترتيب العكسي للـ dependencies (view، وبعدين lots اللي بتشاور على auctions، وبعدين auctions) وإلا الـ FK هيمنع الـ drop. `GO` مش T-SQL، هو فاصل batches بيفهمه SSMS و sqlcmd.

```
CREATE TABLE dbo.auctions (
    auction_id INT NOT NULL PRIMARY KEY, auction_name VARCHAR(60) NOT NULL,
    auction_date DATE NOT NULL, city VARCHAR(40) NOT NULL);
INSERT INTO dbo.auctions VALUES
    (23154, 'Home & Kitchen Liquidation', '2026-03-28', 'Phoenix, AZ'),
    (23166, 'Pallets Liquidation',        '2026-03-28', 'Phoenix, AZ');

CREATE TABLE dbo.lots (
    auction_id INT NOT NULL REFERENCES dbo.auctions (auction_id),
    lot VARCHAR(10) NOT NULL,          -- keeps leading zeros
    title NVARCHAR(300) NOT NULL,
    buyer_id VARCHAR(20) NOT NULL,
    quantity INT NOT NULL,
    hammer_price DECIMAL(10,2) NOT NULL,
    CONSTRAINT pk_lots PRIMARY KEY (auction_id, lot));
```

ليه كده؟

- `DECIMAL(10,2)` للفلوس: الـ FLOAT بيعمل أخطاء تقريب (0.1+0.2).
- `NVARCHAR` للـ title لأن ممكن يبقى فيه Unicode. و `VARCHAR` للـ lot لحفظ الأصفار.
- جدول `auctions` منفصل (dimension صغير) بدل ما نكرر اسم المزاد في كل صف: normalization.
- الـ composite PK بيضمن إن نفس الـ lot مش هيتحمّل مرتين لنفس المزاد (لو شغلت الـ load غلط هيفشل بدل ما يكرر الداتا بصمت).

الجزء 2: Staging + BULK INSERT

```
CREATE TABLE #stage (lot VARCHAR(10), title NVARCHAR(300), buyer_id VARCHAR(20),
                     quantity INT, hammer_price DECIMAL(10,2));

BULK INSERT #stage FROM 'C:\Users\Public\auction-inventory\home_kitchen_results_clean.csv'
WITH (FORMAT = 'CSV', FIRSTROW = 2, CODEPAGE = '65001', TABLOCK);
INSERT INTO dbo.lots SELECT 23154, lot, title, buyer_id, quantity, hammer_price FROM #stage;
```

```
TRUNCATE TABLE #stage;
-- same for pallets with auction_id 23166
```

- الـ CSV مفيهوش عمود auction_id، فبنحمّله في **staging table** مؤقت (`#` = temp table) بنفس شكل الملف، وبعدين نضيف الـ auction_id في الـ INSERT...SELECT.
- `FORMAT='CSV'` : بيفهم الـ quotes (الـ titles اللي فيها فاصلة زي `"Acrylic Sashiko Stencil, Embroidery..."`). من غيره الفاصلة جوه الـ title هتكسر الأعمدة.
- `FIRSTROW=2` : تخطى الـ header. `CODEPAGE='65001'` : UTF-8. `TABLOCK` : lock للجدول بيسرّع الـ bulk load.

> **بدايل للـ BULK INSERT**
>
> (1) **SSMS Import Flat File wizard**: سهل بس مش reproducible. (2) **pandas + SQLAlchemy**: بيشتغل من Python وبيتفادى مشكلة صلاحيات الـ service account. (3) **OPENROWSET(BULK...)** اخترنا BULK INSERT لأنه script واحد بيتشغل بضغطة وبيوري T-SQL.

الجزء 3: الـ view v_lots

ده القلب. الـ CTE الأول `base` :

```
WITH base AS (
    SELECT l.auction_id, a.auction_name, a.auction_date, l.lot,
           REPLACE(REPLACE(REPLACE(l.title, '&#34;', '"'), '&#38;', '&'), '&amp;', '&') AS title,
           l.buyer_id, l.quantity, l.hammer_price,
           PATINDEX('%retail $%', l.title) AS rpos
    FROM dbo.lots AS l JOIN dbo.auctions AS a ON a.auction_id = l.auction_id
),
parsed AS (
    SELECT b.*, CASE WHEN rpos > 0 THEN SUBSTRING(title, rpos + 8, 20) END AS rtext
    FROM base AS b
)
```

> **Worked example: lot 28237**
>
> title = `(Retail $1,121.00) Lanbo 24 Inch Freestanding Electric Range...`
>
> 1. `PATINDEX('%retail $%', title)` = 2 (الموضع اللي بيبدأ منه "Retail $"). بيطابق case-insensitive بسبب الـ collation.
> 2. `rpos + 8` : طول "retail $" هو 8 حروف (6 + مسافة + $)، فبنقفز على الكلمة ونقف عند أول رقم.
> 3. `SUBSTRING(title, 10, 20)` = `1,121.00) Lanbo 24 I` (20 حرف).

وبعدين الـ `CROSS APPLY` في آخر الـ view:

```
CROSS APPLY (SELECT TRY_CAST(
      REPLACE(LEFT(rtext, PATINDEX('%[^0-9.,]%', rtext + 'x') - 1), ',', '')
      AS DECIMAL(10,2)) AS retail_price) AS r
```

تكملة المثال

4. `PATINDEX('%[^0-9.,]%', rtext + 'x')`: بيدوّر على **أول حرف مش رقم ولا نقطة ولا فاصلة**. في `1,121.00) Lanbo...` ده القوس (في الموضع 9.
5. `LEFT(rtext, 9 - 1)` = `1,121.00`.
6. `REPLACE(..., ',', '')` = `1121.00`.
7. `TRY_CAST(... AS DECIMAL(10,2))` = **1121.00**.

و `'x' +` ليه؟ لو الرقم واصل لآخر الـ 20 حرف ومفيش حرف "غريب" بعده، `PATINDEX` هترجع 0 و `LEFT(..., -1)` هيرمي error. بإضافة 'x' بنضمن إن فيه دايماً حرف يوقف عنده. وبالنسبة لـ `White... ($69) (Retail` الناتج `69` → 69.00.

ليه الحل ده؟

- **ليه CROSS APPLY؟** عشان نديّ الناتج اسم (`retail_price`) ونستخدمه في أعمدة تانية في نفس الـ SELECT (`discount_to_retail` و `retail_tier`). من غيره هنكرر التعبير الطويل 3 مرات.
- **ليه TRY_CAST مش CAST؟** لو النص مش رقم يرجّع NULL بدل ما يوقف الـ view كله بـ error.
- **ليه CTEs متتابعة (base → parsed)؟** كل خطوة بتبني على اللي قبلها وتتقري من فوق لتحت.

الطرق البديلة

(1) **CTE مقابل subquery:** نفس النتيجة، بس الـ CTE أسهل قراءة وبتتسمى وتتستخدم أكتر من مرة (subquery متداخل 3 مستويات يبقى كابوس). مفيش فرق أداء كبير هنا؛ SQL Server بيعامل الـ CTE كـ inline زي الـ subquery. (2) **Regex في SQL Server:** مفيش regex حقيقي في الإصدارات القديمة (الـ Express عندي بيعتمد على PATINDEX و LIKE)، عشان كده الـ pattern المحدود `[^0-9.,]`. (3) **تحميل retail_price كعمود جاهز من Python** وبالتالي الـ SQL يبقى بسيط؛ ده أنضف، بس بنفقد إن الـ SQL يبقى self-contained وقابل للتدقيق. اخترنا الـ SQL parsing عشان كل المنطق في مكان واحد.

الـ entities

الـ `REPLACE` الثلاثية بتحوّل `"` لـ `"` و `&` و `&` لـ `&`. دي ما بتأثرش على الـ retail لأن الـ `rpos` بيتحسب من `l.title` الأصلي، بس بتخلّي الـ titles في Power BI مقروءة.

category و price_band و retail_tier

```
CASE
  WHEN title LIKE 'Pallet of%'     THEN 'Pallets (mixed)'
  WHEN title LIKE '%Range Hood%'   THEN 'Range Hoods'
  WHEN title LIKE '%NACS%' OR ... THEN 'Automotive & EV'
  ... (Refrigeration & Ice, Heating, Laundry, Kitchen Appliances,
       Plumbing & Bath, Electronics, Hardware & Electrical)
  ELSE 'Home, Toys & Misc'
END AS category
```

- الـ exports **مفيهاش category**، فعملنا rule-based classification من كلمات في الـ title. أول WHEN يطابق هو اللي بيكسب، فالترتيب مهم: "Range Hood" لازم قبل "Range"/"Oven" عشان متتصنفش Kitchen Appliances.
- الـ `ELSE` هو سلة "Home, Toys & Misc" (15 lot): أي حاجة معرفتش أصنفها.
- الـ `price_band` على الـ hammer: ≤5، 6-20، 21-50، 51-100، أكتر من 100. وبتبدأ بـ "1. " و "2. " عشان الترتيب الأبجدي يطلع صح في SQL وPower BI من غير sort column.

- الـ `retail_tier` على الـ retail: أقل من 100، 100-249، 250-499، 500+، و NULL لو مفيش retail.

> **حدود الـ rule-based categories**
>
> التصنيف heuristic: كلمة زي `%Light%` ممكن تمسك حاجات مش إضاءة (مثلاً "Flashlight" أو أي title فيه "light" كجزء كلمة)، و `%Toyota%` بتحط أي حاجة بالاسم ده في Automotive. فمتعاملش مع الـ category كحقيقة. هي أداة عرض مفيدة، ولازم تقولها كده في الـ interview. مثال على حجم الأثر: Hardware & Electrical فيها 32 lot بس 7 فقط فيهم retail، فالـ recovery بتاعها (15.4%) مبني على 7 lots.

الـ sanity check في الآخر

```
SELECT COUNT(*) AS lots, SUM(hammer_price) AS hammer_total,
       COUNT(retail_price) AS lots_with_retail, SUM(retail_price) AS retail_total
FROM dbo.v_lots;   -- 168 | 3710.02 | 120 | 27838.65
```

`COUNT(retail_price)` (على عمود) بيتجاهل الـ NULLs، عكس `COUNT(*)`. عشان كده بيدّيك 120 مش 168.

5.2 Query 01: ملخص كل مزاد

```
-- Business question: What did each auction sell, and for how much?
-- Lots, hammer total, median / mean / max hammer, unique buyers, and how many lots never moved
-- past the $5 opening bid. GROUPING SETS adds the combined row.
USE Auction_Inventory;

WITH med AS (
    SELECT DISTINCT auction_name,
           PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY hammer_price) OVER (PARTITION BY auction_name) AS median_hammer,
           PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY hammer_price) OVER ()                          AS median_all
    FROM dbo.v_lots
)
SELECT COALESCE(l.auction_name, 'COMBINED')                 AS auction,
       COUNT(*)                                             AS lots,
       SUM(l.hammer_price)                                  AS hammer_total,
       CASE WHEN GROUPING(l.auction_name) = 1 THEN MAX(m.median_all) ELSE MAX(m.median_hammer) END AS median_hammer,
       CAST(AVG(l.hammer_price) AS DECIMAL(10,2))           AS mean_hammer,
       MAX(l.hammer_price)                                  AS max_hammer,
       COUNT(DISTINCT l.buyer_id)                           AS unique_buyers,
       SUM(CASE WHEN l.hammer_price <= 5 THEN 1 ELSE 0 END)  AS lots_at_opening_bid,
       CAST(100.0 * SUM(CASE WHEN l.hammer_price <= 5 THEN 1 ELSE 0 END) / COUNT(*) AS DECIMAL(5,1)) AS
pct_at_opening_bid
FROM dbo.v_lots AS l
LEFT JOIN med AS m ON m.auction_name = l.auction_name
GROUP BY GROUPING SETS ((l.auction_name), ())
ORDER BY CASE WHEN l.auction_name IS NULL THEN 1 ELSE 0 END, hammer_total DESC;
```

> **ليه كتبناه كده؟**
>
> عايزين في جدول واحد: لكل مزاد + صف "COMBINED" إجمالي: عدد lots، الإجمالي، الـ median والـ mean والـ max، عدد المشترين، وكام lot ما عداش الـ $5. المدير بيسأل "إحنا بنبيع كام وبكام" وده الجدول اللي بيجاوب.

بيعمل إيه؟

- **CTE `med`:** بيحسب الـ median. `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY hammer_price)` هي دالة T-SQL **window فقط** (لازم `OVER`)، مش aggregate، فمنقدرش نحطها مع GROUP BY مباشرة. فبنحسبها كـ window مرة `PARTITION BY auction_name` (median لكل مزاد) ومرة `OVER ()` (median للكل)، و `SELECT DISTINCT` بيقلل الصفوف المتكررة لصف واحد لكل مزاد.
- **`GROUP BY GROUPING SETS ((l.auction_name), ())`:** مجموعتين تجميع: حسب المزاد، و `()` = إجمالي الكل. بيطلّع صفوف المزادين + صف الإجمالي في query واحد (بديل لـ UNION ALL).
- **`GROUPING(l.auction_name) = 1`:** بتقول لك إن الصف ده هو صف الإجمالي (الـ NULL اللي ظهر فيه ناتج من التجميع، مش قيمة NULL حقيقية). بنستخدمها لاختيار `median_all` في صف الإجمالي. وفي `COALESCE(l.auction_name,'COMBINED')` بنسمّي الصف ده.
- `MAX(m.median_hammer)`: الـ median قيمة واحدة لكل مزاد، فـ MAX مجرد "خد القيمة" عشان نرضي الـ GROUP BY.
- `SUM(CASE WHEN hammer_price <= 5 THEN 1 ELSE 0 END)`: conditional count (pattern شائع). والنسبة بـ `100.0 * ... /` `COUNT(*)`: الـ `100.0` العشرية ضرورية، لأن القسمة على INT كانت هتقصّ الناتج لرقم صحيح.
- الـ `ORDER BY CASE WHEN auction_name IS NULL THEN 1 ELSE 0 END` بيحط صف الإجمالي آخر، وبعدها `hammer_total DESC`.

النتيجة الحقيقية (متراجعة)

auction	lots	hammer	median	mean	max	buyers	at $5	%
Home & Kitchen	162	3535.02	5.0	21.82	240.01	37	95	58.6
Pallets	6	175.00	30.0	29.17	50.00	4	1	16.7
COMBINED	168	3710.02	5.0	22.08	240.01	41	96	57.1

مثال الـ median بالإيد: الـ Pallets فيها 6 lots مرتبة: 5, 10, 20, 40, 50, 50. العدد زوجي، فالـ median هو متوسط الوسطين (20 و 40) = **30**. وده `PERCENTILE_CONT` (بيعمل interpolation). في الـ 168 lot، 96 منهم بـ $5، فالـ lot رقم 84 و 85 بعد الترتيب الاتنين $5، والـ median = 5. لاحظ إن الـ mean (22.08) أكبر من الـ median بأربع مرات: **right-skewed** بسبب كام جهاز غالي.

ملحوظة: 37 + 4 = 41 يعني مفيش buyer اشترى من المزادين الاتنين. بس متعتمدش على الجمع بإيدك: الـ `COUNT(DISTINCT buyer_id)` على الكل هي اللي بتحسم لأن buyer مشترك كان هيتعدّ مرتين في الجمع.

ليه اخترنا الحل ده؟

عشان نطلع كل حاجة في query واحد ومن غير UNION ALL أو temp tables. الـ GROUPING SETS مقروء ومضمون إن الإجمالي محسوب من نفس الـ rows.

الطرق البديلة

- **الـ median:** (أ) `PERCENTILE_DISC(0.5)` بيرجّع قيمة فعلية من الداتا بدل interpolation (في الـ pallets كان هيطلع 20 مش 30)؛ (ب) حل يدوي بـ `ROW_NUMBER()` و `COUNT()` و متوسط الصفين الأوسطين: أطول وأسهل تغلط فيه؛ (ج) `MEDIAN()` في DAX أو `median()`. في pandas: دول أسهل بكتير لو التحليل برا SQL.
- **الإجمالي:** `UNION ALL` بين query بـ GROUP BY و query من غير، أو `WITH ROLLUP`. الـ GROUPING SETS أوضح لما تعمل مجموعتين بس.
- **الـ CTE مقابل subquery:** كان ممكن نحط الـ median كـ subquery مترابط (correlated) في الـ SELECT. الـ CTE + JOIN أسرع وأقرأ.

5.3 Query 02: توزيع الأسعار

```
-- Business question: Where do lots actually land on price?
-- Lots and hammer dollars per price band; shows how much of the revenue comes from the few big lots.
USE Auction_Inventory;

SELECT price_band,
       COUNT(*)                                                                  AS lots,
       CAST(100.0 * COUNT(*) / SUM(COUNT(*)) OVER () AS DECIMAL(5,1))            AS pct_of_lots,
       SUM(hammer_price)                                                         AS hammer_total,
       CAST(100.0 * SUM(hammer_price) / SUM(SUM(hammer_price)) OVER () AS DECIMAL(5,1)) AS pct_of_hammer
FROM dbo.v_lots
GROUP BY price_band
ORDER BY price_band;
```

ليه كتبناه كده؟

عايزين نشوف: الـ lots بتتركز فين، والفلوس بتيجي منين. غالباً العدد مركّز في الأسعار الواطية والفلوس في الأعلى.

بيعمل إيه؟

`GROUP BY price_band` بيدّيك عدد وإجمالي لكل band. السحر في `SUM(COUNT(*)) OVER ()` : ده **aggregate جوه window**: الـ `COUNT(*)` بيتحسب بعد الـ GROUP BY (عدد كل band)، وبعدين `SUM(...) OVER ()` بيجمع الـ counts كلها = 168 من غير partition، فنقسم عليه ونطلع النسبة. نفس الفكرة في الـ `SUM(SUM(hammer_price)) OVER ()` للدولارات.

النتيجة (متراجعة)

band	lots	% lots	hammer	% hammer
1. $5	96	57.1	480.00	12.9
2. $6-20	27	16.1	375.01	10.1
3. $21-50	24	14.3	774.00	20.9
4. $51-100	14	8.3	1048.00	28.2
5. Over $100	7	4.2	1033.01	27.8

القراءة: **57% من الـ lots بتجيب 13% بس من الفلوس**. و 21 lot (النطاقين الأخيرين = 12.5% من الـ lots) بتجيب حوالي 56% من الدولارات (28.2 + 27.8). ده مثال على الـ Pareto / skew.

الطرق البديلة

(أ) CTE بيحسب الإجماليات وبعدين `CROSS JOIN` مع إجمالي منفصل: أطول. (ب) Subquery `(SELECT SUM(...) FROM v_lots)` في المقام: بيشتغل لكن بيقرا الجدول مرتين. (ج) في DAX: `DIVIDE([Lots Sold], CALCULATE([Lots Sold], ALL(FactLots)))`. اخترنا الـ window لأنه pass واحد وبيحافظ على الـ grouping.

5.4 Query 03: الـ discount to retail

```
-- Business question: How far below retail do lots sell, and does that change with item size?
-- Only lots whose title advertises a retail price. Discount = 1 - hammer / retail (dollar-weighted).
USE Auction_Inventory;
```

```
SELECT COALESCE(retail_tier, 'ALL RETAIL-PRICED LOTS')                  AS retail_tier,
       COUNT(*)                                                         AS lots,
       SUM(retail_price)                                                AS retail_total,
       SUM(hammer_price)                                                AS hammer_total,
       CAST(100.0 * SUM(hammer_price) / SUM(retail_price) AS DECIMAL(5,1))       AS recovery_pct,
       CAST(100.0 * (1 - SUM(hammer_price) / SUM(retail_price)) AS DECIMAL(5,1)) AS discount_pct
FROM dbo.v_lots
WHERE retail_price IS NOT NULL
GROUP BY GROUPING SETS ((retail_tier), ())
ORDER BY CASE WHEN retail_tier IS NULL THEN 1 ELSE 0 END, retail_tier;
```

ليه كتبناه كده؟

السؤال: "الـ lots بتتباع أد إيه تحت الـ retail، وده بيتغير لو القطعة أغلى؟". فبنقسم على `retail_tier` ونجمع صف إجمالي.

بيعمل إيه؟

- `WHERE retail_price IS NOT NULL` : بنقارن بس الـ lots اللي ليها retail (120). لو مدخلناش الفلتر ده، الـ hammer بتاع الـ 48 lot التانيين هيتحسب على المقام من غير retail ويفضّل الـ discount مبالغ فيه.
- `recovery_pct = 100 * SUM(hammer) / SUM(retail)` و `discount_pct = 100 * (1 - ...)` : نسبة **مجموع على مجموع** (dollar-weighted).
- `COALESCE(retail_tier, 'ALL RETAIL-PRICED LOTS')` مع GROUPING SETS: نفس تكتيك query 01.

النتيجة (متراجعة)

tier	lots	retail	hammer	recovery %	discount %
Under $100	20	1674.54	212.50	12.7	87.3
$100-249	72	10682.26	1055.01	9.9	90.1
$250-499	16	6150.94	1045.50	17.0	83.0
$500+	12	9330.91	1007.01	10.8	89.2
ALL	120	27838.65	3320.02	11.9	88.1

حساب الإجمالي بإيدك: 3320.02 ÷ 27838.65 = 0.1193 → 11.9 سنت لكل retail دولار. لو عملت **متوسط الـ discount لكل lot** (unweighted) الرقم هيطلع مختلف، لأن الـ lots الرخيصة هتأثر بنفس وزن الغالية. ده مش خطأ، بس لازم تعرف أنهي واحد بتعرضه. احنا اخترنا dollar-weighted لأنه بيجاوب "لو اشتريت كل الـ 120 lot بكام كنت هتدفع مقابل retail كام".

صدق في التفسير

الـ $250-499 tier فيها 16 lot بس، فالـ 17% دي ممكن تتحرك بسبب lot أو اتنين. وعدم وجود pattern واضح (الأغلى مش بالضرورة أرخص نسبياً) مع عينة بالحجم ده متعتبرهوش اتجاه حقيقي.

الطرق البديلة

(أ) متوسط `AVG(discount_to_retail)` (unweighted): أسهل وبيجاوب سؤال مختلف. (ب) الـ median للـ discount: مقاوم للـ outliers. (ج) `WITH ROLLUP` بدل GROUPING SETS. (د) حساب الـ tiers في Power BI بدل SQL: أقل ثبات لأن الـ logic هيبقى مخفي في الـ report.

5.5 Query 04: أعلى الـ lots

```
-- Business question: Which lots drove the money?
-- Top 10 lots by hammer price with category, advertised retail and discount; RANK() keeps ties honest.
USE Auction_Inventory;

WITH ranked AS (
    SELECT RANK() OVER (ORDER BY hammer_price DESC) AS price_rank,
           lot, category, LEFT(title, 70) AS title, hammer_price, retail_price,
           CAST(100.0 * discount_to_retail AS DECIMAL(5,1)) AS discount_pct
    FROM dbo.v_lots
)
SELECT * FROM ranked
WHERE price_rank <= 10
ORDER BY price_rank, lot;
```

ليه؟

"إيه الـ lots اللي جابت الفلوس؟". وبنستخدم `RANK()` عشان نتعامل مع التعادل بأمانة.

بيعمل إيه؟

الـ CTE بيحسب الـ `RANK() OVER (ORDER BY hammer_price DESC)` لكل lot، والـ SELECT الخارجي بيفلتر `price_rank <= 10` . ليه محتاجين CTE؟ لأن **الـ window functions مينفعش تتحط في WHERE مباشرة** (الـ WHERE بيتنفذ قبل الـ window). فبنلفّها في CTE وبنفلتر برا.

`LEFT(title, 70)` بتقصّر الـ title للعرض. والـ `discount_pct = discount_to_retail * 100` من الـ view.

النتيجة، وفخ التعادل

rank	lot	category	hammer	retail	disc%
1	28142	Refrigeration & Ice	240.01	969.00	75.2
2	28112	Refrigeration & Ice	168.00	669.99	74.9
3	28123	Kitchen Appliances	161.00	359.99	55.3
4	28108	Refrigeration & Ice	134.00	533.00	74.9
5	28124 / 28125 / 28144	Plumbing, Refrig, Refrig	110.00	three-way tie	
8	28074	Heating	95.00	409.00	76.8
9	28073	Kitchen Appliances	91.00	141.45	35.7
10	28104 / 28115 / 28140	Refrigeration	85.00	three-way tie	

"Top 10" رجّعت **12 صف** لأن فيه تعادلين. و RANK بتقفز (بعد 5-5-5 جاي 8 مش 6). ده بالظبط معنى "ties honest". في الداشبورد بنعرض "Top 8 lots" (الصفوف اللي رانكها ≤ 8)، فيهم 7 أجهزة (5 ثلاجة/heater، food warmer، ice) و toilet واحد (28124).

الطرق البديلة (window مقابل self join)

- `ROW_NUMBER()` : بيرجّع 10 صفوف بالظبط لكن بيكسر التعادل عشوائياً (ظلم للـ lot الخامس عشر مثلاً).
- `DENSE_RANK()` : من غير قفزات (1,2,3,4,5,5,5,6...).
- `TOP 10 WITH TIES` : يوصل لنفس نتيجة RANK بدون CTE: `SELECT TOP 10 WITH TIES ... ORDER BY hammer_price DESC` . أقصر، بس مفيش عمود rank.
- **Self join:** `SELECT a.* FROM v_lots a WHERE (SELECT COUNT(*) FROM v_lots b WHERE b.hammer_price > a.hammer_price) < 10`: بيشتغل بس أبطأ ($O(n^2)$) وأصعب قراءة. الـ window function بتعدّي على الداتا مرة واحدة. ده سؤال interview كلاسيكي: "window أم self join؟" والإجابة: window.

Query 05 5.6: تركّز المشترين

```
-- Business question: Is the buying crowd concentrated or spread out?
-- Spend per buyer, share of total, and running cumulative share (window functions over the CTE).
USE Auction_Inventory;

WITH spend AS (
    SELECT buyer_id, COUNT(*) AS lots_won, SUM(hammer_price) AS spend
    FROM dbo.v_lots
    GROUP BY buyer_id
)
SELECT TOP 10
       RANK() OVER (ORDER BY spend DESC)                                        AS buyer_rank,
       buyer_id, lots_won, spend,
       CAST(100.0 * spend / SUM(spend) OVER () AS DECIMAL(5,1))                 AS pct_of_hammer,
       CAST(100.0 * SUM(spend) OVER (ORDER BY spend DESC, buyer_id ROWS UNBOUNDED PRECEDING)
            / SUM(spend) OVER () AS DECIMAL(5,1))                               AS cumulative_pct,
       COUNT(*) OVER ()                                                         AS total_buyers
FROM spend
ORDER BY spend DESC, buyer_id;
```

ليه؟

لو 2-3 مشترين بياخدوا أغلب الفلوس، فالمزاد "هش": انسحابهم يغيّر الأسعار. وده بيوصل لـ "إيه مدى تنافسية السوق".

بيعمل إيه؟

- `spend` CTE : صف لكل buyer فيه عدد lots وإجمالي الإنفاق (GROUP BY buyer_id). هنا الـ grain اتغيّر من lot لـ buyer: خطوة مهمة تتفهم.
- `RANK() OVER (ORDER BY spend DESC)` : ترتيب الإنفاق.
- `spend / SUM(spend) OVER ()` : نسبة كل buyer من الكل (window بدون partition).
- **الـ running total:** `SUM(spend) OVER (ORDER BY spend DESC, buyer_id ROWS UNBOUNDED PRECEDING)` يجمع من أول صف لحد الصف الحالي. الـ `ROWS` مهم: من غيره الـ default هو `RANGE` اللي بيعامل الصفوف المتعادلة كمجموعة واحدة ويدّيها نفس الـ cumulative. و `buyer_id` كـ tie-breaker بيخلّي الترتيب ثابت.
- `COUNT(*) OVER () + TOP 10` = 41 (إجمالي المشترين: بيتحسب قبل ما TOP يقص).

النتيجة (متراجعة)

rank	buyer	lots	spend	% of $	cumulative %
1	29214	19	835.00	22.5	22.5
2	195520	4	399.00	10.8	33.3
3	109944	22	371.00	10.0	43.3
4	13240	22	370.00	10.0	53.2
5	18025	21	282.51	7.6	60.8

مثال حساب: 835 ÷ 3710.02 = 22.5%. أول 5 مجموعهم 835+399+371+370+282.51 = 2257.51 → 60.8%. لاحظ إن الـ buyer رقم 2 اشترى 4 lots بس بـ $399 (lots غالية)، وأرقام 3 و 4 و 5 اشتروا 21-22 lot لكل واحد بأسعار واطية (غالباً الـ $5 lots: ممكن يكونوا resellers بياخدوا بالجملة، لكن ده تخمين، مش مثبت من الداتا).

الطرق البديلة

(أ) الـ cumulative بـ **self join** (`JOIN ... ON b.spend >= a.spend` وبعدين SUM): بطيء ومعقد، والـ window أنضف. (ب) `NTILE` أو `PERCENT_RANK` لو عايز deciles. (ج) مقياس تركّز رسمي زي **HHI** (مجموع مربعات الحصص). (د) `DENSE_RANK` لو عايز ranks من غير فجوات. و في Power BI: Top 5 Buyer Share بـ TOPN (هنشرحه في DAX).

5.7 Query 06: الـ category و الـ "margin"

```
-- Business question: Which categories carry the resale margin?
-- Per category: lots, hammer, average lot price, and for retail-priced lots the dollar spread
-- between advertised retail and what the lot hammered for (the room a reseller has to work with).
USE Auction_Inventory;

SELECT category,
       COUNT(*)                                                                  AS lots,
       SUM(hammer_price)                                                         AS hammer_total,
       CAST(AVG(hammer_price) AS DECIMAL(10,2))                                  AS avg_hammer,
       COUNT(retail_price)                                                       AS lots_with_retail,
       SUM(retail_price)                                                         AS retail_total,
       SUM(CASE WHEN retail_price IS NOT NULL THEN retail_price - hammer_price END) AS retail_spread,
       CAST(100.0 * SUM(CASE WHEN retail_price IS NOT NULL THEN hammer_price END)
            / NULLIF(SUM(retail_price), 0) AS DECIMAL(5,1))                      AS recovery_pct
FROM dbo.v_lots
GROUP BY category
ORDER BY hammer_total DESC;
```

الاسم مضلل: ده مش margin

اسم الملف `06_category_margin.sql` بس اللي بيحسبه هو **الفجوة بين الـ retail المعلن والـ hammer** (retail spread)، مش margin. الـ margin محتاج cost وإيراد إعادة بيع: وده مش في الداتا. ده الـ "room" اللي الـ reseller ممكن يشتغل فيه في أحسن الأحوال.

بيعمل إيه؟

- GROUP BY category وبيحسب lots و hammer و avg.
- `COUNT(retail_price)` = عدد الـ lots اللي فيها retail (بتتجاهل NULL).
- `SUM(CASE WHEN retail_price IS NOT NULL THEN retail_price - hammer_price END)` : الفجوة على الـ lots اللي ليها retail **بس**. (الـ CASE من غير ELSE بيرجّع NULL والـ SUM بيتجاهله.)
- `recovery_pct` : بسط = hammer على الـ lots اللي فيها retail بس (like-for-like)، مقام = `NULLIF(SUM(retail_price),0)` . الـ `NULLIF` بتحوّل الصفر لـ NULL فالقسمة تدّي NULL مش error. كده category الـ Pallets (مفيهاش retail) بتطلع NULL.

النتيجة (متراجعة)

category	lots	hammer	w/ retail	retail	spread	recov %
Refrigeration & Ice	14	1249.51	13	7233.97	5994.46	17.1
Kitchen Appliances	13	567.00	10	2544.43	2042.43	19.7
Heating	30	435.50	30	4935.70	4500.20	8.8
Automotive & EV	14	328.01	13	2629.25	2306.24	12.3
Plumbing & Bath	19	310.00	18	4030.62	3725.62	7.6
Hardware & Electrical	32	251.00	7	819.89	693.89	15.4
Range Hoods	12	150.00	12	3369.47	3219.47	4.5
Electronics	12	136.50	9	1229.25	1107.75	9.9
Laundry	5	102.50	4	495.14	412.64	16.7
Pallets (mixed)	2	90.00	0	NULL	NULL	NULL
Home, Toys & Misc	15	90.00	4	550.93	515.93	6.4

مثال: Refrigeration: 13 lot فيهم retail بإجمالي 7233.97، الـ hammer على نفس الـ 13 = 7233.97 - 5994.46 = 1239.51 (والـ category hammer الكلي 1249.51 لأن فيه lot رقم 14 من غير retail بـ $10). الـ recovery = 1239.51 ÷ 7233.97 = 17.1%. لاحظ إن `avg_hammer` بيتحسب على كل الـ lots، لكن `recovery_pct` على الـ lots اللي فيها retail بس.

الفرق بين "أكبر فجوة دولارية" و "أرخص نسبياً": Refrigeration فجوتها الدولارية أكبر ($5,994) لأن الأجهزة غالية، لكن recovery 17.1% يعني مش الأرخص نسبياً. Range Hoods أرخص نسبياً (4.5 سنت لكل دولار) بس فجوتها $3,219 أقل. سؤالين مختلفين.

الطرق البديلة

(أ) كان ممكن نعمل `WHERE retail_price IS NOT NULL` في subquery بدل CASE جوه الـ SUM، لكن كنا هنفقد الـ lots من غير retail في الـ count والـ hammer. (ب) `PIVOT` لو عايز categories كأعمدة (مش مفيد هنا). (ج) `AVG(discount_to_retail)` لكل category: unweighted. (د) نحسبه في DAX بـ Recovery Rate و Retail Spread مع category على المحور: وده اللي الداشبورد بيعمله فعلاً.

Power BI (6

6.1 الموديل

الموديل بسيط جداً (وده مقصود): جدولين بس.

- **FactLots**: 168 صف، جاي من الـ view `dbo.v_lots` عن طريق Import.
- **_Measures**: جدول فاضي بيحمل الـ measures (عادة الـ underscore بتخليه أول واحد في الـ Fields pane).

مفيش relationships لأن كل حاجة (auction_name و category و price_band) جوه الـ view بالفعل: **flat fact table**. ده بيشتغل في مشروع بالحجم ده، بس في موديل أكبر هتعمل star schema بـ (DimAuction, DimCategory, DimBuyer, DimDate) dimension tables.

> **ليه الـ view مش جداول منفصلة؟**
>
> المنطق (retail و category) متكتب مرة واحدة في SQL ومحدش بيعيده في Power BI. لو حد غيّر قاعدة category، يتغيّر في مكان واحد.

Power Query 6.2

```
let
    Source = Sql.Database(".\SQLEXPRESS", "Auction_Inventory"),
    v_lots = Source{[Schema = "dbo", Item = "v_lots"]}[Data],
    #"Changed Types" = Table.TransformColumnTypes(v_lots, { ... hammer_price Currency.Type, ... }),
    #"Added Lot Title" = Table.AddColumn(#"Changed Types", "Lot Title", each
        let t = if Text.StartsWith([title], "(") then Text.Trim(Text.AfterDelimiter([title], ")")) else [title]
        in if Text.Length(t) > 55 then Text.Start(t, 52) & "..." else t, type text)
in #"Added Lot Title"
```

- بنختار الـ view بالـ navigation (`Source{[Schema="dbo",Item="v_lots"]}`) بدل ما نكتب native SQL query: كده الـ **query folding** شغال والتحويلات بتتنفذ في SQL Server.
- `Lot Title` عمود محسوب: بيشيل البادئة `(Retail $...)` ويقص لـ 55 حرف عشان الجدول يبقى مقروء.
- **Gotcha:** الـ M بيفرق بين الحروف الكبيرة والصغيرة (`Text.StartsWith` case-sensitive) وبرضه بيحتاج انتباه في اسم الـ instance.

6.3 الـ 15 measure واحد واحد

أساس مهم: الـ measure في DAX بيتحسب **حسب الـ filter context**: يعني نفس الـ measure بيدّي رقم مختلف لو حطيته في card (كل الداتا) أو في bar chart لكل category (كل bar فلتر مختلف) أو مع slicer المزاد. إنت مبتكتبش loop، إنت بتكتب تعريف والـ engine بيطبقه على كل سياق.

#	Measure	DAX	value (all data)
1	Lots Sold	COUNTROWS (FactLots)	168
2	Hammer Total	SUM (FactLots[hammer_price])	$3,710
3	Avg Hammer	AVERAGE (FactLots[hammer_price])	$22.08
4	Median Hammer	MEDIAN (FactLots[hammer_price])	$5.00
5	Top Lot	MAX (FactLots[hammer_price])	$240
6	Unique Buyers	DISTINCTCOUNT (FactLots[buyer_id])	41
7	Lots at Opening Bid	CALCULATE (COUNTROWS (FactLots), FactLots[hammer_price] <= 5)	96
8	Opening Bid Share	DIVIDE ([Lots at Opening Bid], [Lots Sold])	57.1%
9	Retail Value	SUM (FactLots[retail_price])	$27,839
10	Hammer on Retail-Priced Lots	CALCULATE ([Hammer Total], NOT ISBLANK (FactLots[retail_price]))	$3,320
11	Recovery Rate	DIVIDE ([Hammer on Retail-Priced Lots], [Retail Value])	11.9%
12	Discount to Retail	IF (NOT ISBLANK ([Retail Value]), 1 - [Recovery Rate])	88.1%
13	Retail Spread	[Retail Value] - [Hammer on Retail-Priced Lots]	$24,519
14	Top Buyer Share	DIVIDE (MAXX (VALUES (FactLots[buyer_id]), [Hammer Total]), [Hammer Total])	22.5%
15	Top 5 Buyer Share	DIVIDE (SUMX (TOPN (5, VALUES (FactLots[buyer_id]), [Hammer Total]), [Hammer Total]), [Hammer Total])	60.8%

(الـ Retail Spread الكلي = 27,838.65 - 3,320.02 = $24,518.63: حسبته من الأرقام المتراجعة.)

1-6: الـ aggregations البسيطة

- **Lots Sold** = `COUNTROWS(FactLots)` : بيعدّ الصفوف الظاهرة بعد الفلاتر. لأن الـ grain = lot، الصفوف = lots. وعلى slicer "Pallets" هيرجّع 6.
- **Hammer Total** = `SUM` : أساس بقية الـ measures.
- **Avg Hammer** و **Median Hammer**: الاتنين موجودين عشان الفرق بينهم هو الحكاية (22.08 مقابل 5.00). `MEDIAN` هنا هي **aggregation function** في DAX، مش محتاج PERCENTILE.
- **Top Lot** = `MAX` .
- **Unique Buyers** = `DISTINCTCOUNT` : بيعدّ القيم المختلفة. مش COUNT (اللي هيدّي 168).

7-8: Opening Bid

- **Lots at Opening Bid**: `CALCULATE` بتعدّل الـ filter context: بتضيف شرط `hammer_price <= 5` فوق الفلاتر الموجودة. ده الشرط الـ (5) hardcoded، وهو غلط لأي مزاد opening bid بتاعه مختلف.
- **Opening Bid Share**: `DIVIDE` بدل `/` . الـ DIVIDE لما المقام صفر أو BLANK بترجّع BLANK بدل ما تطلّع Infinity أو error. وده الأمان.

9-13: عائلة الـ retail (الأهم)

- **Retail Value**: `SUM(retail_price)` . الـ SUM بتتجاهل BLANK، فالـ 48 lot اللي مفيهمش retail مبيدخلوش الحساب.
- **Hammer on Retail-Priced Lots**: ده الـ measure اللي بيمنع الغلطة: مينفعش نقسم Hammer Total (كل الـ 168) على Retail Value (120 lot بس): ده هيعمل apples-to-oranges. الـ CALCULATE بتفلتر الـ hammer لنفس الـ 120 lot: $3,320.

- **Recovery Rate**: hammer ÷ retail على نفس الـ lots = 11.9%. ده الـ "cents per retail dollar".
- **Discount to Retail** = `IF(NOT ISBLANK([Retail Value]), 1 - [Recovery Rate])` : من غير else، فبترجّع BLANK لو مفيش retail. ده **مقصود**: عشان category زي Pallets متظهرش "100% discount" (1 = BLANK - 1 لأن BLANK بيتعامل 0 في الحساب).
- **Retail Spread** = الفرق الدولاري. لو Retail Value هي BLANK، التفريق بيرجّع BLANK (مش بالضرورة: BLANK - BLANK = BLANK).

فخ BLANK() < 3

في DAX، `BLANK()` بيتقارن كأنه صفر. يعني `BLANK() < 3` بترجّع **TRUE**. لو عملت conditional formatting أو IF زي `IF([Discount to Retail] < 0.3, ...)` هتتصنّف "Low discount" retail (Pallets) اللي مفيهاش categories، الحل: `IF(NOT ISBLANK([Discount to Retail]) && [Discount to Retail] < 0.3, ...)`. وكمان `[x] = 0` بترجّع TRUE لما x هو BLANK. استخدم `ISBLANK` لما تقصد "مفيش قيمة" و `=0` لما تقصد صفر فعلاً. وده فرق مهم: `==` بيفرّق بينهم.

14-15: تركّز المشترين

- **Top Buyer Share** = `DIVIDE(MAXX(VALUES(buyer_id), [Hammer Total]), [Hammer Total])`.
  1. `VALUES(FactLots[buyer_id])` : جدول مؤقت بالـ buyers المميزين في الـ filter context الحالي.
  2. `MAXX` بتلف على الجدول ده، وجوّه كل iteration بتحسب `[Hammer Total]`. ده **context transition**: الـ measure لما يتنفذ جوه iterator بيتحوّل الصف الحالي (buyer واحد) لفلتر، فبيطلع إنفاق ذلك الـ buyer.
  3. الـ MAXX بترجّع أكبر إنفاق (835)، والـ DIVIDE بيقسم على الإجمالي (3710) = 22.5%.
- **Top 5 Buyer Share**: `TOPN(5, VALUES(buyer_id), [Hammer Total])` بيجيب أعلى 5 buyers حسب الإنفاق، و `SUMX` بتجمع إنفاقهم = 2,257.51 ÷ 3710.02 = 60.8%. (في الداشبورد 60.8%، ومكتوبة "about 61%" في التوثيق.)

الـ measures دي context-sensitive

لو حطيت slicer على category، الـ Top Buyer Share بيبقى "أكبر buyer جوه الـ category دي من إنفاق الـ category" مش من الكل. دي مش غلطة، بس لازم تعرف تفسّرها. ولو عايز الإجمالي دايماً، هتلف الـ denominator بـ `CALCULATE(..., ALL(FactLots))`. وفي ties في `TOPN` ممكن يرجّع أكتر من 5 صفوف لو فيه تعادل على المركز الخامس.

بدايل DAX

(أ) `DIVIDE` مقابل `/` مع `IF` : DIVIDE أقصر وأأمن. (ب) `CALCULATE(..., KEEPFILTERS(...))` لو عايز تحافظ على فلتر موجود على نفس العمود: الـ `CALCULATE(..., hammer_price <= 5)` بيستبدل أي فلتر سابق على `hammer_price`. (ج) `NOT ISBLANK(retail_price)` مقابل `BLANK() <> retail_price` : التاني بيعامل 0 زي BLANK أحياناً. (د) **Calculated columns بدل measures**: مش محسوب حسب الـ context، يضخّم الـ model. الـ measure أنسب للنسب لأن النسبة مش بتتجمع (مجموع النسب مش نسبة المجاميع).

6.4 الـ visuals وليه

Page 1: Liquidation Auction Results (فيه slicer اسمه Auction)

- **4 cards:** Lots Sold (168)، Hammer Total ($3,710)، Median Lot Price ($5.00)، Sold at $5 Opening Bid (57.1%). كارت واحد لكل رقم رئيسي: أول حاجة العين بتشوفها.
- **Bar chart "Hammer dollars by category":** أفقي لأن أسماء الـ categories طويلة، مرتب تنازلي.
- **Column chart "Lots by hammer price band":** بيوري شكل التوزيع. الـ band بترتيب "1. 2. 3." عشان يترتب صح. 7، 14، 24، 27، 96
- **Table "Top 8 lots by hammer price":** Lot, Lot Title, Category, Hammer Total, Retail Value, Discount to Retail. الجدول للتفاصيل اللي مينفعش تتعرض في chart.

Page 2: What Lots Sell For vs. Retail

- **4 cards:** Advertised Retail Value ($27,839)، Discount to Retail (88.1%)، Unique Buyers (41)، Top 5 Buyers' Share ($) (60.8%).
- **Bar "Cents on the retail dollar by category":** الألوان أخضر/برتقالي/أحمر (green = الأرخص للشراء). Range Hoods 4.5%، Home/Toys Kitchen Appliances 19.7%... Plumbing 7.6%، 6.4% أغلى. الـ recovery بتتعرض بدل الـ discount لأن "سنت لكل دولار" أبسط للمقارنة.
- **Column "by retail price tier":** 12.7%، 9.9%، 17.0%، 10.8%.
- **Bar "Top 10 buyers by spend":** أعلى buyer 29214 بـ $835.
- **Key Findings text box:** الخلاصة مكتوبة بالكلام، "All numbers verified in SQL (sql/01 to 06)".

قواعد الـ design اللي اتبعناها

Dark theme وLight theme (ملفات JSON في reports/themes)، لون واحد للتركيز، عناوين بتقول الاستنتاج مش بس اسم الـ metric، وأرقام كبيرة في الـ cards. كل visual بيجاوب سؤال واحد.

انتبه

اللون الأخضر/الأحمر في recovery: الأخضر = "أرخص للمشتري" (recovery واطي). ده ممكن يتلخبط مع منطق "أخضر = كويس" في تقارير أخرى، فاللجند والعنوان بيوضحوا ده.

7) إزاي تبني المشروع من الصفر: Checklist

1. نزّل الـ results من LocalAuctions.com (Excel). متلمسهمش: خليهم في data/raw/.
2. افتحهم بـ openpyxl وشوف الصفوف الأولى. حدد صف الـ header الحقيقي.
3. اكتب `load_results` تنضف وتحافظ على الـ lot و buyer_id كنص.
4. اكتب `RETAIL_RE` مع `re.IGNORECASE`. جرّبه على 5 titles (فيهم $69 من غير سنت و $1,121.00 بفاصلة وواحد بـ "REtail").
5. اطبع عدد الـ lots اللي لقاها: لازم 120 من 168.
6. اكتب CSVs بـ `encoding="utf-8"` صريح.
7. انسخ الـ CSVs لـ C:\Users\Public\auction-inventory\.
8. اكتب `00_setup_database.sql`: database، auctions، lots، staging، BULK INSERT، view. شغّله، وتأكد: 168 | 3710.02 | 120.
9. اكتب 01 لـ 06 واحد واحد. بعد كل query قارن بالـ Python summary.
10. Power BI: Get Data → SQL Server → .\SQLEXPRESS, Auction_Inventory → اختار `v_lots` (Import).
11. اعمل جدول `_Measures` وضيف الـ 15 measure. اتحقق: Lots Sold = 168، Hammer = 3,710، Recovery = 11.9%.
12. ابني الصفحتين. حط slicer للـ Auction. جرّب تختار Pallets وشوف إن الـ BLANK بيظهر مظبوط.
13. طبّق الـ theme. خد screenshots.
14. اكتب README بالـ limitations: عينة صغيرة، static، مفيش cost.
15. راجع كل رقم في الـ README مقابل SQL قبل ما تنشر.

> **Validation checkpoints**
>
> 168 lots → $3,710.02 → 120 retail lots → $27,838.65 retail → $3,320.02 hammer on them → 41 buyers → 96 at $5. لو أي رقم منهم اختلف، وقّف وفتّش قبل ما تكمل.

8) Gotchas

الفخ	الإصلاح
الـ regex من غير IGNORECASE بيطلع 119 مش 120 ("lot 28031 "REtail")	دايماً case-insensitive على نص بشري، وقارن العدّ بين Python و SQL.
BULK INSERT بيفشل "Access is denied"	الـ service account مش بيدخل Downloads: استخدم C:\Users\Public\...
الأصفار في الـ lot بتضيع (0094 → 94)	VARCHAR في SQL، و dtype=str في pandas.
الفاصلة جوه الـ title بتكسر الأعمدة	`FORMAT='CSV'` + الـ CSV مقتبس.
الـ encoding على Windows	اكتب CSV بـ utf-8 و `CODEPAGE='65001'`.
PATINDEX بترجع 0 فـ LEFT(-1) بيرمي error	الـ `+ 'x'` حارس.
COUNT(*) مقابل COUNT(col)	الثاني بيتجاهل NULL: 168 مقابل 120.
القسمة الصحيحة (100 / 168 = 0)	استخدم `100.0`.
Hammer Total ÷ Retail Value (apples-to-oranges)	استخدم Hammer on Retail-Priced Lots.
BLANK() < 3 بترجّع TRUE في DAX	`NOT ISBLANK` الأول.
RANK بيدّي أكتر من 10 صف (12 في query 04) بسبب التعادل	مقصود. استخدم ROW_NUMBER لو عايز 10 بالظبط.
الـ $5 opening bid مكتوب hardcoded (في SQL و DAX)	غيّره لو المزاد ليه opening مختلف.
اسم الملف 06 بيقول margin	هو retail spread. متقولش "margin" في الـ interview.
الـ category بالـ LIKE: أول match بيكسب	الترتيب مهم، وراجع الـ misclassification.
الـ hammer مش شامل buyer's premium (15%) ولا tax (9.1%) في الـ template)	اللي المشتري بيدفعه فعلاً أعلى من الـ hammer.
الـ README بيقول إن Python بيعمل categorize	في الكود الفعلي الـ category بتتحسب في SQL فقط.

Interview Prep (9

15 سؤال. كل سؤال: السؤال بالإنجليزي، إجابة مثالية وصادقة بأرقام المشروع الحقيقية، وبعدها **الاستراتيجية** بالمصري. اتدرّب تقول الإجابات بصوتك بكلامك انت، مش تحفظها حرفياً.

Q1. Walk me through this project in two minutes.

> I analyzed 168 lots from two liquidation auctions that ran in Phoenix on March 28, 2026. I cleaned the Excel exports with Python, loaded them into SQL Server Express, built a view that parses the advertised retail price out of the lot titles and classifies each lot, and then built a two-page Power BI dashboard on top of it. The headline: total hammer was $3,710.02, the median lot sold for $5, 57% of lots never went above the $5 opening bid, and the 120 lots that advertised a retail price sold for about 11.9 cents per retail dollar. Buying was concentrated, with the top five of 41 buyers taking 60.8% of the dollars. I want to be clear about limits: it is two auctions on one day, and there is no cost data, so I measure hammer against advertised retail, not margin.

الاستراتيجية: قدّم القصة بترتيب: سؤال، داتا، pipeline، نتيجة، حدود. ادّي 3 أرقام بس (168، 11.9 سنت، 60.8%) وقول الـ limitation بنفسك قبل ما حد يسأل. ده بيبني ثقة. متقولش 'margin' أبداً.

Q2. Why did you parse the retail price in SQL as well as in Python?

> The pipeline does both on purpose. Python parses the retail price with a regex for an independent cross-check summary, while the SQL view is what Power BI actually reads, so the business logic lives in one auditable place. In SQL I used PATINDEX and SUBSTRING because I did not use a regex engine there, with TRY_CAST so a malformed price becomes NULL instead of breaking the view. Having two implementations let me compare counts, and that comparison is what exposed the case-sensitivity bug.

الاستراتيجية: الإجابة الذكية: مش تردد، ده قرار. ذكر إن عندك مصدرين = validation. لو سألوك 'مش تكرار؟' قول إن التكرار هنا فايدته cross-check. وبعدين وصّل للـ bug story.

Q3. Tell me about a bug you found in this project.

> One lot, 28031, had the title '(REtail $119.99)' with an unusual capital E. A regex pattern for 'Retail' without re.IGNORECASE missed it, so I counted 119 retail-priced lots instead of 120 and understated retail value by $119.99. The way to catch it is to reconcile the counts between layers: the SQL view uses a case-insensitive collation and reports 120. The fix was the IGNORECASE flag. The lesson I took was to make any pattern over human-typed text case-insensitive by default and to reconcile row counts across every layer of the pipeline.

الاستراتيجية: STAR: Situation (عدّين مختلفين)، Task (أعرف ليه)، Action (قارنت الـ layers ودوّرت على الفرق)، Result (120 والـ 28031 lot). رقم صغير ($119.99) بيوري إنك مهتم بالدقة حتى لو الأثر صغير. متبالغش في أهمية الـ bug.

Q4. What is the difference between a median and a mean here, and why does it matter?

> The median lot price is $5.00 while the mean is $22.08. 96 of the 168 lots, 57.1%, sold at the $5 opening bid, so the typical lot is the opening bid, but a handful of appliances, up to $240.01, pull the mean up. The distribution is right-skewed. If I reported only the mean I would suggest a typical lot costs $22, which is wrong for most lots. In T-SQL I computed the median with PERCENTILE_CONT as a window function, and in DAX with the MEDIAN function.

الاستراتيجية: اشرح skew بجملة: 'نص الـ lots بخمسة دولار وكام جهاز غالي بيرفعوا المتوسط'. وجاهز تقول ليه PERCENTILE_CONT (interpolation) مقابل PERCENTILE_DISC.

Q5. Why did you use PERCENTILE_CONT with OVER, and what are the alternatives?

> In T-SQL PERCENTILE_CONT is only available as a window function, so to get a median per auction next to GROUP BY aggregates I computed it in a CTE with PARTITION BY and joined it back, using DISTINCT to collapse duplicates. For the six-lot pallet auction, with sorted prices 5, 10, 20, 40, 50, 50, it interpolates between 20 and 40 and returns 30, whereas PERCENTILE_DISC would return 20. Alternatives are a ROW_NUMBER approach picking the middle rows, which is longer and easier to get wrong, or computing it outside SQL in pandas or DAX.

الاستراتيجية: سؤال تقني كلاسيكي. اذكر: DISTINCT trick، window only، الفرق بين CONT و DISC بمثال رقمي (30 مقابل 20). كده بتوري إنك فاهم مش حافظ.

Q6. Window function versus self join: when do you use each? Show an example from the project.

> I use window functions whenever I need a row-level value next to a group-level or running value, because they scan the data once and are easier to read. In query 05 I compute each buyer's share with SUM(spend) OVER () and the running cumulative share with SUM(spend) OVER (ORDER BY spend DESC, buyer_id ROWS UNBOUNDED PRECEDING). The self-join equivalent joins the table to itself on spend greater than or equal to, which is quadratic and harder to maintain. I specified ROWS rather than the default RANGE so tied spenders are not collapsed into one cumulative value, and added buyer_id as a tie-breaker for deterministic results.

الاستراتيجية: اذكر ROWS مقابل RANGE: ده اللي بيفرق بين مرشح عادي ومرشح قوي. ومتنساش تقول إن الـ self join ممكن يكون مقبول لو الداتا صغيرة، بس الـ window أنضف.

Q7. Why RANK() instead of ROW_NUMBER() in the top-lots query?

> Because ties are real. Three lots hammered at $110 and three at $85, so a top-10 by RANK returns 12 rows, which is honest, while ROW_NUMBER would arbitrarily drop some tied lots. RANK also skips numbers after a tie, which is why rank 5 is followed by rank 8. The dashboard shows the top 8 lots, which is exactly the rows with rank at most 8. If I needed exactly ten rows I would use ROW_NUMBER with a documented tie-breaker, or TOP 10 WITH TIES.

الاستراتيجية: ادّي الرقم 12 صف بدل 10 كدليل إنك لاحظت التعادل. وتواضع: 'لو المطلوب 10 بالظبط كنت غيرت'.

Q8. CTE or subquery? Why a view for the model?

> Functionally they are equivalent here, SQL Server inlines both, so the choice is readability. I used chained CTEs in the view because each step, cleaning the title, locating the price, parsing it, builds on the previous one and reads top to bottom. A CTE can also be referenced multiple times. I put the logic in a view so the retail parsing, categories and price bands are defined once and Power BI just imports them, instead of re-implementing business rules in the report.

الاستراتيجية: متقولش 'CTE أسرع'. قول 'أقرأ'. وربط الـ view بمبدأ single source of truth.

Q9. Explain Top Buyer Share. Why does the measure use MAXX over VALUES?

> VALUES(buyer_id) returns the distinct buyers in the current filter context, and MAXX iterates over them evaluating [Hammer Total] for each. Because a measure referenced inside an iterator triggers context transition, each iteration evaluates that single buyer's total, so MAXX returns the largest buyer spend, $835, and DIVIDE by total hammer gives 22.5%. Top 5 Buyer Share uses TOPN(5, ...) with SUMX, giving 60.8%. These are context-sensitive: under a category slicer they show the leader within that category, which I documented as a pitfall.

الاستراتيجية: أهم كلمتين: filter context و context transition. لو مش متأكد من الشرح، ارسم على ورقة buyers وكل واحد بإجماليه. وقول الـ pitfall بنفسك.

Q10. What is the BLANK() less than 3 pitfall, and how did you guard against it?

> In DAX, BLANK is treated as zero in comparisons, so BLANK() < 3 returns TRUE. A conditional format such as 'discount below 30%' would flag categories with no retail data, such as the mixed pallets, as low-discount. My Discount to Retail measure intentionally returns BLANK when Retail Value is BLANK, using IF without an else branch, and any threshold logic should first check NOT ISBLANK. I also built Recovery Rate with DIVIDE, which returns BLANK instead of an error or infinity on a zero or BLANK denominator.

الاستراتيجية: سؤال DAX بيفرّز اللي شغّال فعلاً. اذكر مثال حقيقي: Pallets (mixed) مفيهاش retail. وبيّن إنك متعمّد BLANK مش مصادفة.

Q11. What can and can't you conclude from two auctions?

> I can describe what happened at these two Phoenix auctions on March 28, 2026: most lots cleared at the $5 opening bid, retail-priced lots sold at around 11.9 cents per advertised retail dollar, and a few buyers took most of the dollars. I cannot say these patterns hold in general, that any category is reliably cheap, or what a reseller would earn: there are no cost, fee, shipping or resale-price columns, the hammer excludes the buyer's premium, and the retail figure is the seller's advertised number, not verified market value. Some category conclusions rest on very few lots, for example Hardware & Electrical has 32 lots but only 7 with a retail price. I'd treat this as a descriptive baseline and a hypothesis generator, not as evidence for an investment decision.

الاستراتيجية: ده أهم سؤال business. الجواب القوي هو اللي بيحدد الحدود بنفسه بأرقام. متقولش 'التحليل بيثبت'. استخدم: describe، can't، small n، no cost، generalize. وبعدها اقترح الخطوة الجاية (جمع مزادات أكتر + cost).

Q12. Is an 88.1% discount to retail a reliable number?

> It is a correct calculation on a limited basis. It is dollar-weighted: $3,320.02 hammer divided by $27,838.65 advertised retail across the 120 lots that advertise a price gives 11.9% recovery, so 88.1% discount. The caveats are that the retail value is the seller's advertised figure, 48 of 168 lots have no retail price and are excluded, condition varies, and hammer excludes the buyer's premium. I would present it as 11.9 cents per advertised retail dollar rather than as 'real savings'.

الاستراتيجية: اتكلم بـ 'cents per retail dollar' عشان تبيّن إنك فاهم إن الـ discount ده مقارنة بسعر معلن. اذكر الـ 48 اللي مستبعدين.

Q13. How would you improve or extend this project?

> First, more auctions across dates and locations to get a meaningful sample and test whether patterns persist. Second, add cost data, buyer's premium, tax and resale prices so I could estimate real margin. Third, replace the keyword category rules with a maintained mapping table and review misclassifications. Fourth, remove the hardcoded $5 opening bid by storing it per auction, and write the cleaned data to SQL directly from Python to avoid the BULK INSERT file-permission step. Finally, add automated checks that assert the row counts, 168 lots and 120 retail-priced lots, after each load.

الاستراتيجية: رتّب التحسينات: داتا أكتر، cost، تصنيف أحسن، إزالة automation، hardcoding. وحط 'automated checks' لأنها بتربط بقصة الـ bug.

Q14. Tell me about a time you realized you were wrong or over-claimed. (Behavioral)

> While documenting this project I noticed one of my file names, 06_category_margin.sql, over-claimed: it computes the gap between advertised retail and hammer, not a margin, because the data has no cost. I corrected the wording and now describe it as retail spread, the room a reseller could work with. I think being precise about what the data supports matters more than having an impressive headline.

الاستراتيجية: HR بيدوّر على self-awareness. الإجابة دي حقيقية ومن المشروع. بدل كلام عام، وري إنك اكتشفت المبالغة وصلّحتها. نبرة هادية ومن غير دفاع.

Q15. Why did you choose this project, and what did you learn? (HR)

> I wanted a realistic messy dataset instead of a clean tutorial file. The Excel exports had junk header rows, prices buried in free-text titles, and HTML entities, so I had to build the whole path from raw file to dashboard. I learned to reconcile counts across layers, to be explicit about the grain, to choose the right denominator when comparing hammer to retail, and to state a project's limits openly. It also taught me to validate every number in the README against a query before publishing.

الاستراتيجية: قول دوافع عملية: داتا وسخة حقيقية. وبعدين 3-4 دروس محددة. وأي HR يحب جملة 'validate قبل ما أنشر'.

أرقام لازم تحفظها

168 lots	$3,710.02 hammer	median $5	mean $22.08	96 at $5 (57.1%)
120 retail lots	$27,838.65 retail	$3,320.02 hammer	11.9% / 88.1%	41 buyers
top buyer 22.5% ($835)	top 5: 60.8%	Range Hoods 4.5%	Refrigeration $5,994.46	bug: 119 vs 120, lot 28031

> **نصيحة أخيرة من المنتور**
>
> أعد بناء المشروع مرة بإيدك من غير ما تبص على الكود: ابدأ من الـ checklist في الفصل 7. أول ما توصل لـ 168 | 3710.02 | 120 وانت كاتبه بنفسك، يبقى فاهمه فعلاً. وفي أي interview، الصراحة في الـ limitations بتكسّبك مصداقية أكتر من أي رقم كبير.